Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»
Кафедра инженерной психологии и эргономики
Учебная дисциплина «Современные языки программирования»

ОТЧЕТ
по лабораторной работе №8
«Разработка приложения с применением событий,
анимации»
Вариант 4

Подготовила: Студент гр. 410902
Большакова М.В.

Проверила: Семерник О.Д.

Минск 2026

Цель: выполнить разработку приложения с использованием: привязки объектов, подключением звука, API интерфейсов, токенов, электронной почты, парсинга.

Теоретические сведения по теме лабораторной работы:

1. Что такое событийно-ориентированное программирование?

Это событийная модель, где действия пользователя (клики, движения мыши, клавиши) порождают события, на которые программа реагирует через обработчики событий (callback-функции).

2. Как в tkinter реагировать на нажатие клавиши?

Для этого можно использовать метод bind(), например: entry.bind("<Return>", on_enter).

3. Как реализуется цикл анимации?

Используется метод after(), который вызывает функцию через заданный интервал времени, имитируя кадры и движения. Например: root.after(50, animate).

4. Как вызвать функцию с задержкой?

Для этого применяется метод after(), который вызывает функцию через определенный интервал времени.

5. Что такое after()?

Это метод для анимации, вызывающий нужную функцию через заданный промежуток времени.

6. Какие библиотеки используются для воспроизведения звука?

Применяются модули pygame.mixer (простая и удобная библиотека) и playsound (минималистичный способ).

7. Как использовать playsound?

Нужно импортировать функцию и передать в нее название файла: from playsound import playsound, а затем playsound('alert.mp3').

8. Что такое API?

Это REST API — интерфейс, через который современные сервисы предоставляют данные.

9. Что такое токен доступа?

Это ключ доступа, который требуется для авторизации в сервисах, предоставляющих данные.

10. Как передать токен в заголовке запроса?

Он передается через словарь headers: headers = {"Authorization": "Bearer <токен>"}, который затем вставляется в метод: response = requests.get(url, headers=headers).

11. Что такое JSON?

Это формат, в котором API часто возвращают данные, и который легко обрабатывать.

12. Как загрузить JSON в Python?

Нужно импортировать модуль json и использовать метод .json() для ответа сервера, например: data = response.json().

13. Как спарсить температуру из API погоды?

Путем обращения к ключам словаря: print(data["main"]["temp"]).

14. Что такое requests.get().json()?

В методичке это используется как цепочка для получения данных: requests.get() делает запрос по ссылке, а метод .json() преобразует полученный ответ.

15. Как отправить e-mail с Python?

С помощью модуля `smtplib`. Программа подключается к серверу `smtplib.SMTP()`, включает шифрование `server.starttls()`, входит по логину и паролю `server.login()`, и отправляет письмо `server.send_message()`.

16. Какие модули используются для отправки писем?

Используются модули `smtplib` и `email.mime.text`.

17. Что такое MIMEText?

Это функция из модуля `email.mime.text`, в которую передается текст сообщения (например, `MIMEText("Привет!")`) для формирования письма.

18. Как выполнить долгую задачу без блокировки GUI?

Долгие задачи выполняются в фоновом потоке с помощью модуля `threading`. Делается это так: `threading.Thread(target=background_task).start()`.

19.В чём разница между threading и after()?

- `threading` запускает задачу в фоновом потоке, чтобы не блокировать интерфейс при долгих процессах (запросы API, парсинг).

- `after()` просто вызывает функцию через заданный интервал времени (используется для анимации)

20.Как сохранить пользовательские данные в файл?
Настройки и историю удобно сохранять в формате `.json` с помощью команды `with open("settings.json", "w") as f:`, а затем записывать туда данные через `json.dump(data, f)`.

Вариант 4
Отправка писем с GUI и логированием
Форма: адрес, тема, текст. Отправка через SMTP, лог ошибок и статуса. Сохранение истории.

Рабочий код программы:

```
import smtplib
from email.mime.text import MIMEText
import tkinter as tk
import json
import datetime


def send_email():
  resipient = email_entry.get()
  tema = entry_tema.get()
  text_email = message.get("1.0", tk.END)

  sender = "margobolshakova49@gmail.com"
  password = "dytx priv vnxd bort"

  try:
    msg = MIMEText(text_email)
    msg["Subject"] = tema
    msg["From"] = sender
    msg["To"] = resipient
    server = smtplib.SMTP("smtp.gmail.com", 587)
    server.starttls() #шифрование
    server.login(sender, password)
    server.send_message(msg)
    print("Письмо отправлено")
```

```
    with open("log.txt", "a", encoding="UTF-8") as f: #"a" - append(добавить)
     now = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
     f.write(f"[{now}] отправлено на адресок: {resipient}\n")

  except Exception as e:
    print(f"Ошибка при отправлке сообщения: {e}")

    with open("log.txt", "a", encoding="UTF-8") as f:
     now = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
     f.write(f"[{now}] не отправлено, ошибочка: {e}\n")

root = tk.Tk()
root.title("Почтальён")
root.geometry("600x520")

#кому
tk.Label(root, text="Кому(gmail)", font=("JMH Typewriter", 20)).pack(pady=20)
email_entry = tk.Entry(root, width=40,bg= "grey" )
email_entry.pack(pady=5)

#тема
tk.Label(root, text="Тема: ", font=("JMH Typewriter", 20)).pack(pady=20)
entry_tema = tk.Entry(root, width=40, bg= "grey")
entry_tema.pack(pady=5)

#текст письма
tk.Label(root, text="Текст письма: ", font=("JMH Typewriter", 20)).pack(pady=20)
message = tk.Text(root, height=5, wrap="word", bg= "grey")
message.pack()

tk.Button(root, text="Отправить", command= send_email).pack()


root.mainloop()
```

Интерфейс программы:

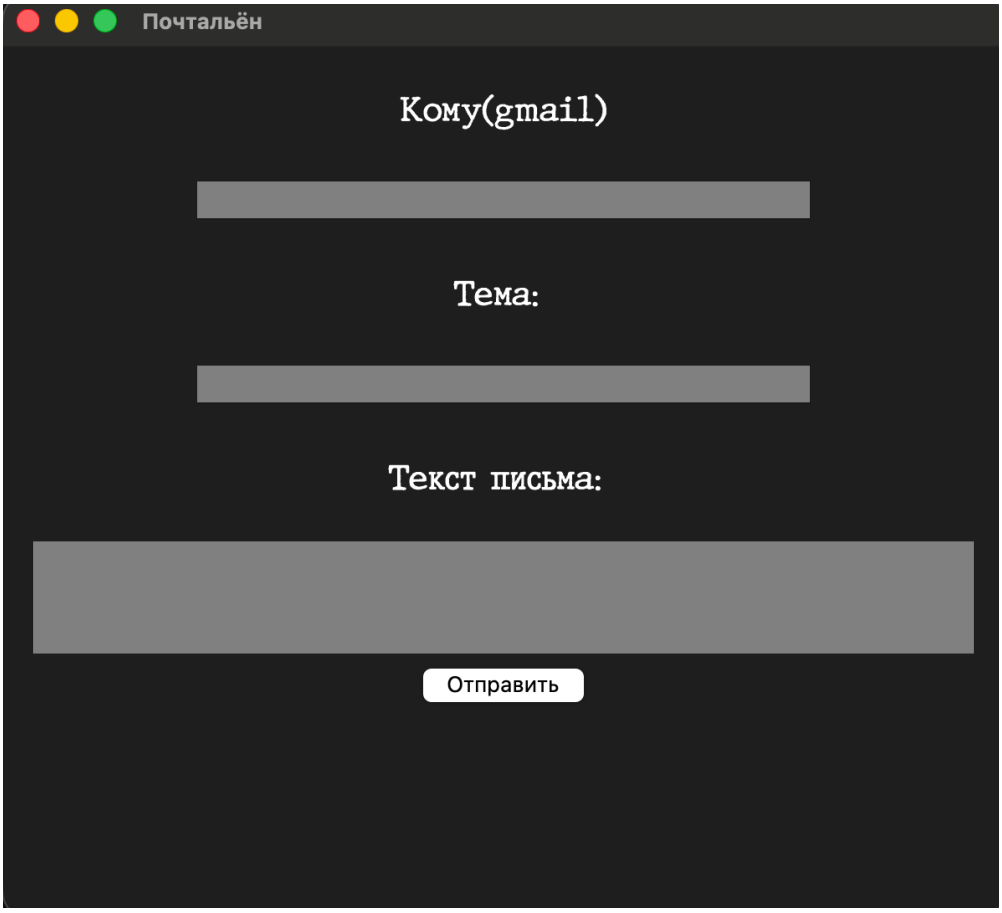

Рисунок 1 – Интерфейс программы

Рисунок 2 – Ввод данных в интерфейсе

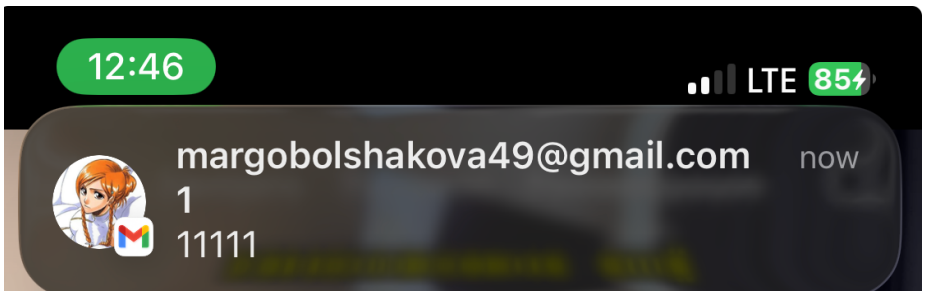

Рисунок 3 – Приход уведомления на введённую почту(реализация SMTP)

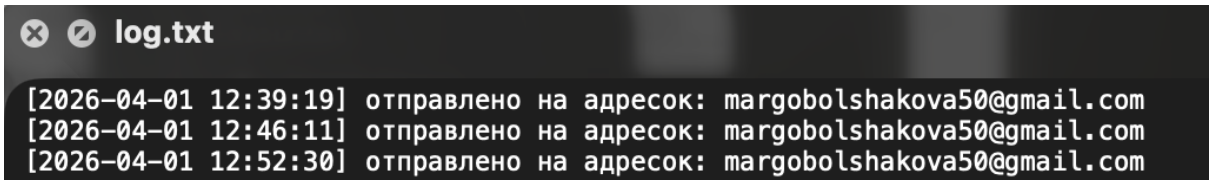

Рисунок 4 – Сохранение истории

Вывод: в ходе выполнения лабораторной работы было разработано графическое приложение «Почтовый клиент» с использованием библиотеки tkinter на основе событийно-ориентированной модели. На практике освоены механизмы формирования и отправки электронных писем по протоколу SMTP с применением встроенных модулей smtplib и email.mime.text, а также настройка защищенного TLS-соединения. Дополнительно реализована система файлового логирования: с помощью конструкции try/except и модуля datetime программа успешно перехватывает ошибки и сохраняет историю статусов отправки с фиксацией точного времени. Цель лабораторной работы достигнута.